

Move Zeroes

Two Pointers · In-place Array Partitioning · Striver TUF+ Revision Notes

O(N) Time

O(1) Space

PROBLEM STATEMENT

Given an integer array `nums`, move all 0's to the end while maintaining the relative order of non-zero elements — in-place, without a copy, for the optimal solution.

Example: `[1,0,2,3,2,0,0,4,5,1] → [1,2,3,2,4,5,1,0,0,0]`

Approach 1 · Brute Force (Temporary Array)

Intuition

Separate non-zero numbers into a temp array while scanning left to right — this automatically preserves their relative order. Copy them back to the front of `nums`, then fill whatever is left with 0.

Approach

- Traverse `nums`; push every `nums[i] != 0` into `temp`.
- Let `nz = temp.size()`. Copy `temp[0..nz-1]` back into `nums[0..nz-1]`.
- Fill `nums[nz .. n-1]` with 0.

KEY OBSERVATION

Scanning strictly left to right means non-zero elements land in `temp` in exactly the order they should appear in the answer — no sorting or extra repositioning is ever needed.

Code (C++)

```
void moveZeroes(vector<int>& nums) {
    int n = nums.size();
    vector<int> temp; // holds non-zero elements

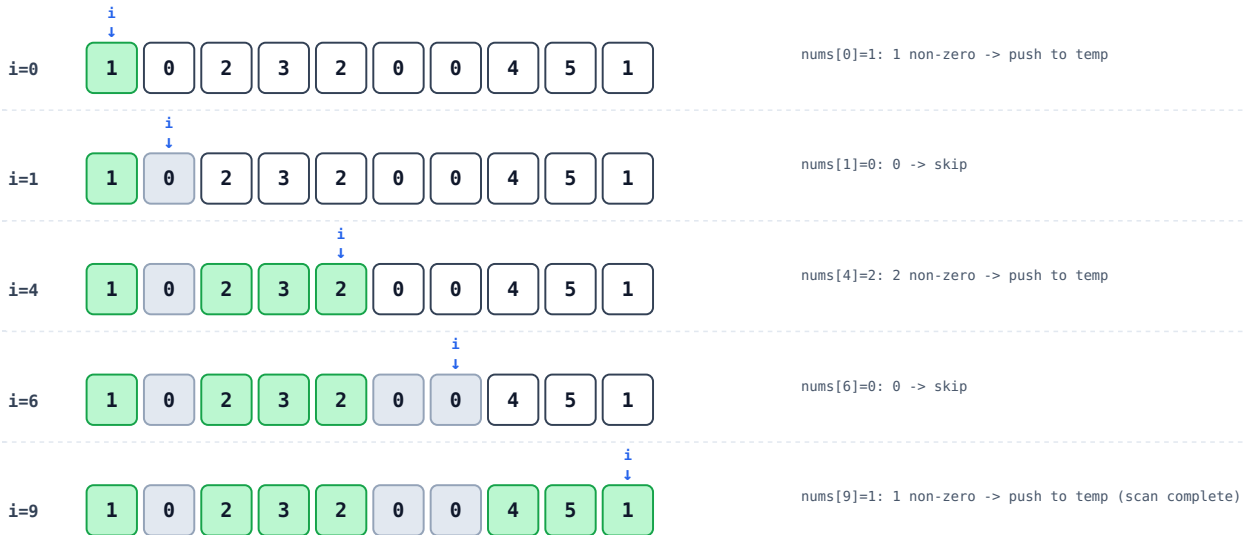
    for (int i = 0; i < n; i++)
        if (nums[i] != 0) temp.push_back(nums[i]);

    int nz = temp.size(); // count of non-zero elements
    for (int i = 0; i < nz; i++) nums[i] = temp[i]; // copy back
    for (int i = nz; i < n; i++) nums[i] = 0; // fill zeroes
}
```

Dry Run

☒ Finalized non-zero ☐ Zero / not placed ☐ Untouched **i ↓ current ptr** **j ↓ placement ptr**

Iteration · scanning `nums` left to right (`i` moves, `temp` grows)



Scan complete — temp array built

temp

1 2 3 2 4 5 1

Non-zero elements collected in order: `temp = [1,2,3,2,4,5,1] → nz = 7`.

After copy-back + zero-fill (final result)



nums[0..6] = temp values, nums[7..9] = 0. Final: [1,2,3,2,4,5,1,0,0,0]

Complexity

Time: $O(2N) \approx O(N)$ Space: $O(N)$

- Building temp: one full pass $\rightarrow O(N)$. Copying back: $O(X)$ where X = count of non-zero elements. Filling zeros: $O(N-X)$. Total $O(N)+O(X)+O(N-X) = O(2N) \approx O(N)$, but with a bigger constant than the optimal solution below.
- temp can hold up to N elements in the worst case (no zeros at all) $\rightarrow O(N)$ auxiliary space.

Approach 2 · Optimal (Two Pointers / In-place Swap)

Intuition

Conveyor belt of boxes, some empty (zero) some full (non-zero), worked by two pointers. i scans every box looking for a non-empty one; j marks the next slot where a non-empty box belongs. On $nums[i] \neq 0$: swap($nums[i]$, $nums[j]$) then $j++$. On zero: just let i move on. Non-zero values get pulled to the front in original order, zeros get pushed back — all in one pass, zero extra space.

Approach

- Init $j = 0$. Traverse with i from 0 to $n-1$.
- If $nums[i] \neq 0$: swap $nums[i] \leftrightarrow nums[j]$, then $j++$.
- If $nums[i] == 0$: do nothing, let i advance.

INVARIANT

Region $[0, j)$ always holds only non-zero elements in original order — that's why a single pass suffices.

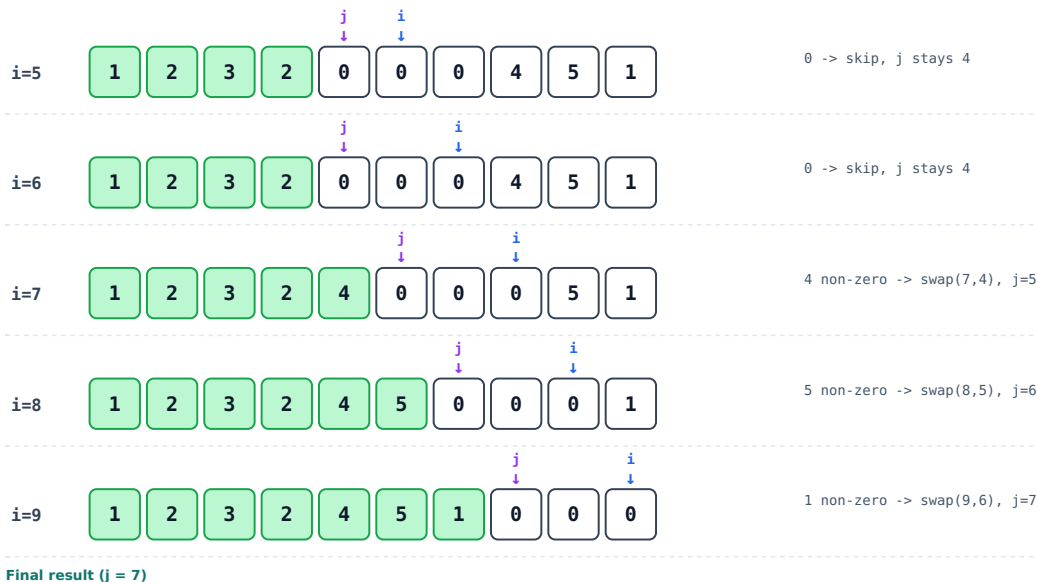
Code (C++)

```
void moveZeroes(vector<int>& nums) {
    int j = 0; // next slot for a non-zero value
    for (int i = 0; i < nums.size(); i++) {
        if (nums[i] != 0) {
            swap(nums[i], nums[j]); // place non-zero at j
            j++; // move destination pointer ahead
        }
        // nums[i] == 0 -> just let i move on, j stays put
    }
}
```

Dry Run · key steps (green = finalized region $[0, j)$)

☒ Finalized non-zero ☐ Zero / not placed ☐ Untouched $i \downarrow$ current ptr $j \downarrow$ placement ptr
Initial ($j = 0$)





Complexity

Time: $O(N)$

Space: $O(1)$

- A single loop runs i from 0 to $n-1$ exactly once, doing $O(1)$ work per index (one comparison, at most one swap + increment) $\rightarrow$ total $O(N)$, no nested loops or re-scans.
- Only two integer variables (i , j) are used and the swap happens in-place $\rightarrow O(1)$ auxiliary space, no data structure that grows with N .

Common Revision Section

Corner Case (both approaches)	What Happens
Empty array	Both approaches do nothing — loops never execute. Safe.
All elements zero	Brute: temp stays empty, whole array filled with 0. Optimal: j never advances, array unchanged.
No zeros present	Brute: temp = full copy, zero-fill loop runs 0 times. Optimal: every step swaps with itself ($i == j$) — array unchanged either way.
Single element	Trivial in both — kept as-is if non-zero, or stays/becomes 0.
Zeros already at the end	Both give the same array back; optimal does harmless self-swaps, brute copies in the same order.
Duplicates / negative numbers	Condition is only $!= 0$ in both — duplicates and negative non-zero values are treated like any other non-zero value.

Pattern Used

Two Pointers — slow/fast in-place partition (one pointer scans, the other marks the boundary of the finalized region). Recognize it whenever a problem asks to *reorder/partition/filter in-place* while preserving relative order, targeting $O(1)$ space. **Similar:** Remove Element, Remove Duplicates from Sorted Array, Segregate Even/Odd, Sort Colors (Dutch National Flag), Partition Array by Pivot.

Tips to Remember

- j never exceeds i — it only advances on a valid non-zero find.
- **Mistake:** incrementing j unconditionally instead of only after a swap — breaks the invariant.
- **Mistake:** shifting elements one-by-one with nested loops ($O(N^2)$) instead of swap-based single pass.
- **Rule of thumb:** "move/group/filter in-place, keep relative order" → think two pointers (write pointer + read pointer). This is a *stable* partition — order within the kept group must be preserved, unlike a quicksort partition.

Quick Revision (30-60 sec)

PROBLEM

Move zeros to end, keep relative order of non-zero elements.

PATTERN

Two Pointers — in-place partition (slow/fast).

KEY IDEA

j = next non-zero slot; swap($nums[i]$, $nums[j]$) when $nums[i] \neq 0$.

COMPLEXITY

Optimal $O(N)/O(1)$. Brute $O(N)/O(N)$.

Approach Comparison

	Main Idea	Time	Space	When to Use
Brute Force	Collect non-zero in temp, copy back, fill rest with zeros.	$O(2N) \approx O(N)$	$O(N)$	Only if extra space is fine, or as a first working solution.
Optimal · Two Pointers	Swap-based single pass with read pointer i and write pointer j .	$O(N)$	$O(1)$	Always preferred — true in-place, ideal for interviews.